

SMART CONTRACT AUDIT REPORT

for

2Factor Finance

Prepared By: Xiaomi Huang

PeckShield

March 16, 2026

Document Properties

Client	Fragments, Inc.
Title	Smart Contract Audit Report
Target	2Factor Finance
Version	1.0
Author	Xuxian Jiang
Auditors	Matthew Jiang, Xuxian Jiang
Reviewed by	Xiaomi Huang
Approved by	Xuxian Jiang
Classification	Public

Version Info

Version	Date	Author(s)	Description
1.0	March 16, 2026	Xuxian Jiang	Final Release
1.0-rc1	February 26, 2026	Xuxian Jiang	Release Candidate #1

Contact

For more information about this document and its contents, please contact PeckShield Inc.

Name	Xiaomi Huang
Phone	+86 183 5897 7782
Email	contact@peckshield.com

Contents

1 Introduction **4**

1.1 About 2Factor Finance 4
1.2 About PeckShield 5
1.3 Methodology 5
1.4 Disclaimer 7

2 Findings **9**

2.1 Summary 9
2.2 Key Findings 10

3 Detailed Results **11**

3.1 Revisited bondCDR Calculation Logic in TrancheHelpers 11
3.2 Incorrect incrementTrancheBal() Logic in PerpHelpers 12
3.3 Enhanced redeemImmature() Logic in BondsManager 13
3.4 Improved Reserve Tranche Data Cloning Logic in PerpHelpers 15
3.5 Revisited Tranche Valuation Calculation Logic 16
3.6 Trust Issue of Admin Keys 17

4 Conclusion **19**

References **20**

1 Introduction

Given the opportunity to review the design document and related smart contract source code of the 2Factor Finance protocol, we outline in the report our systematic approach to evaluate potential security issues in the smart contract implementation, expose possible semantic inconsistencies between smart contract code and design document, and provide additional suggestions or recommendations for improvement. Our results show that the given version of smart contracts can be further improved due to the presence of several issues related to either security or performance. This document outlines our audit results.

1.1 About 2Factor Finance

2Factor Finance is an innovative protocol for perpetual tranches (perp-tranches). The system creates senior (Sr) and junior (Jr) tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes (PerpSr) for lower-volatility exposure or perpetual junior notes (PerpJr) for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The basic information of the audited protocol is as follows:

Table 1.1: Basic Information of The 2Factor Finance Protocol

Item	Description
Client	Fragments, Inc.
Website	https://2factor.finance
Type	EVM Smart Contract
Platform	Solidity
Audit Method	Whitebox
Latest Audit Report	March 16, 2026

In the following, we show the Git repository of reviewed files and the commit hash value used in

this audit.

- https://github.com/fragmentsorg/2factor.git (dc1e08e)

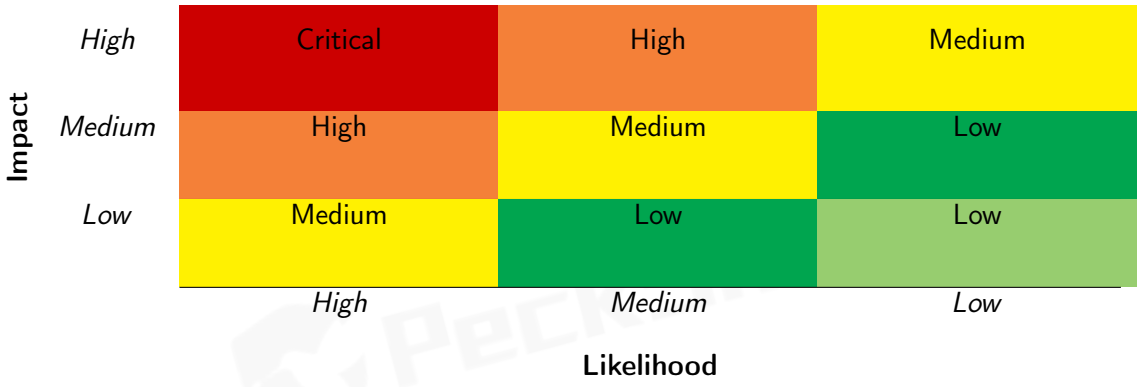
And here is the commit ID after all fixes for the issues found in the audit have been checked in:

- https://github.com/fragmentsorg/2factor.git (0ff7ffb)

1.2 About PeckShield

PeckShield Inc. [9] is a leading blockchain security company with the goal of elevating the security, privacy, and usability of current blockchain ecosystems by offering top-notch, industry-leading services and products (including the service of smart contract auditing). We are reachable at Telegram (https://t.me/peckshield), Twitter (http://twitter.com/peckshield), or Email (contact@peckshield.com).

Table 1.2: Vulnerability Severity Classification



1.3 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [8]:

- Likelihood represents how likely a particular vulnerability is to be uncovered and exploited in the wild;
- Impact measures the technical loss and business damage of a successful attack;
- Severity demonstrates the overall criticality of the risk.

Table 1.3: The Full List of Check Items

Category	Check Item
Basic Coding Bugs	Constructor Mismatch
	Ownership Takeover
	Redundant Fallback Function
	Overflows & Underflows
	Reentrancy
	Money-Giving Bug
	Blackhole
	Unauthorized Self-Destruct
	Revert DoS
	Unchecked External Call
	Gasless Send
	Send Instead Of Transfer
	Costly Loop
	(Unsafe) Use Of Untrusted Libraries
	(Unsafe) Use Of Predictable Variables
	Transaction Ordering Dependence
	Deprecated Uses
Semantic Consistency Checks	Semantic Consistency Checks
Advanced DeFi Scrutiny	Business Logics Review
	Functionality Checks
	Authentication Management
	Access Control & Authorization
	Oracle Security
	Digital Asset Escrow
	Kill-Switch Mechanism
	Operation Trails & Event Generation
	ERC20 Idiosyncrasies Handling
	Frontend-Contract Integration
	Deployment Consistency
	Holistic Risk Management
Additional Recommendations	Avoiding Use of Variadic Byte Array
	Using Fixed Compiler Version
	Making Visibility Level Explicit
	Making Type Inference Explicit
	Adhering To Function Declaration Strictly
	Following Other Best Practices

Likelihood and impact are categorized into three ratings: *H*, *M* and *L*, i.e., *high*, *medium* and *low* respectively. Severity is determined by likelihood and impact and can be classified into four categories accordingly, i.e., *Critical*, *High*, *Medium*, *Low* shown in Table 1.2.

To evaluate the risk, we go through a list of check items and each would be labeled with a severity category. For one check item, if our tool or analysis does not identify any issue, the contract is considered safe regarding the check item. For any discovered issue, we might further deploy contracts on our private testnet and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.3.

In particular, we perform the audit according to the following procedure:

- Basic Coding Bugs: We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- Semantic Consistency Checks: We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
- Advanced DeFi Scrutiny: We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- Additional Recommendations: We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [7], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development. Though some categories used in CWE-699 may not be relevant in smart contracts, we use the CWE categories in Table 1.4 to classify our findings.

1.4 Disclaimer

Note that this security audit is not designed to replace functional tests required before any software release, and does not give any warranties on finding all possible security issues of the given smart contract(s) or blockchain software, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit-based assessment cannot be considered comprehensive, we always recommend proceeding with several independent audits and a public bug bounty program to ensure the security of smart contract(s). Last but not least, this security audit should not be used as investment advice.

Table 1.4: Common Weakness Enumeration (CWE) Classifications Used in This Audit

Category	Summary
Configuration	Weaknesses in this category are typically introduced during the configuration of the software.
Data Processing Issues	Weaknesses in this category are typically found in functionality that processes data.
Numeric Errors	Weaknesses in this category are related to improper calculation or conversion of numbers.
Security Features	Weaknesses in this category are concerned with topics like authentication, access control, confidentiality, cryptography, and privilege management. (Software security is not security software.)
Time and State	Weaknesses in this category are related to the improper management of time and state in an environment that supports simultaneous or near-simultaneous computation by multiple systems, processes, or threads.
Error Conditions, Return Values, Status Codes	Weaknesses in this category include weaknesses that occur if a function does not generate the correct return/status code, or if the application does not handle all possible return/status codes that could be generated by a function.
Resource Management	Weaknesses in this category are related to improper management of system resources.
Behavioral Issues	Weaknesses in this category are related to unexpected behaviors from code that an application uses.
Business Logics	Weaknesses in this category identify some of the underlying problems that commonly allow attackers to manipulate the business logic of an application. Errors in business logic can be devastating to an entire application.
Initialization and Cleanup	Weaknesses in this category occur in behaviors that are used for initialization and breakdown.
Arguments and Parameters	Weaknesses in this category are related to improper use of arguments or parameters within function calls.
Expression Issues	Weaknesses in this category are related to incorrectly written expressions within code.
Coding Practices	Weaknesses in this category are related to coding practices that are deemed unsafe and increase the chances that an exploitable vulnerability will be present in the application. They may not directly introduce a vulnerability, but indicate the product has not been carefully developed or maintained.

2 Findings

2.1 Summary

Here is a summary of our findings after analyzing the design and implementation of the 2Factor Finance protocol. During the first phase of our audit, we study the smart contract source code and run our in-house static code analyzer through the codebase. The purpose here is to statically identify known coding bugs, and then manually verify (reject or confirm) issues reported by our tool. We further manually review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.

Severity	# of Findings	
Critical	0	
High	0	
Medium	1	■
Low	5	■■■■■
Informational	0	
Total	6	

We have so far identified a list of potential issues: some of them involve subtle corner cases that might not be previously thought of, while others refer to unusual interactions among multiple contracts. For each uncovered issue, we have therefore developed test cases for reasoning, reproduction, and/or verification. After further analysis and internal discussion, we determined a few issues of varying severities that need to be brought up and paid more attention to, which are categorized in the above table. More information can be found in the next subsection, and the detailed discussions of each of them are in Section 3.

2.2 Key Findings

Overall, these smart contracts are well-designed and engineered, though the implementation can be improved by resolving the identified issues (shown in Table 2.1), including 1 medium-severity vulnerability and 5 low-severity vulnerabilities.

Table 2.1: Key 2Factor Finance Audit Findings

ID	Severity	Title	Category	Status
PVE-001	Low	Revisited bondCDR Calculation Logic in TrancheHelpers	Coding Practices	Resolved
PVE-002	Low	Incorrect incrementTrancheBal() Logic in PerpHelpers	Business Logic	Resolved
PVE-003	Low	Enhanced redeemImmature() Logic in BondsManager	Business Logic	Resolved
PVE-004	Low	Improved Reserve Tranche Data Cloning Logic in PerpHelpers	Coding Practices	Resolved
PVE-005	Low	Revisited Tranche Valuation Calculation Logic	Business Logic	Resolved
PVE-006	Medium	Trust on Admin Keys	Security Features	Mitigated

Besides recommending specific countermeasures to mitigate these issues, we also emphasize that it is always important to develop necessary risk-control mechanisms and make contingency plans, which may need to be exercised before the mainnet deployment. The risk-control mechanisms need to kick in at the very moment when the contracts are being deployed in mainnet. Please refer to Section 3 for details.

3 Detailed Results

3.1 Revisited bondCDR Calculation Logic in TrancheHelpers

- ID: PVE-001
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `TrancheHelpers`
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

The `2Factor Finance` protocol has a utility `TrancheHelpers` contract to compute a given tranche's value or a parent bond's collateralization/deviation ratio. In the process of examining its logic to compute the collateralization/deviation ratio from a pre-fetched tranche data snapshot, we notice its implementation may be improved.

To elaborate, we show below the implementation of this specific function, i.e., `bondCDR()`. While it properly computes the ratio for a senior tranche, its calculation for a junior tranche does not consider a specific corner case, i.e., `td.valueRatio == ONE`. Specifically, the junior tranche may have `ONE` as its `valueRatio`. In this unlikely corner case, the calculation may need to adjust accordingly to ensure the ratio can always be reliably calculated. Current implementation has a possibility of running into a divide-by-zero issue.

```
38  function bondCDR(
39    ITranche.TrancheData memory td
40  ) internal pure returns (uint256) {
41    if (td.isMature) {
42      return ONE;
43    }
44    return
45      td.isSenior
46        ? ONE.mulDiv(td.trancheRatio, td.valueRatio)
47        : ONE.mulDiv(ONE - td.trancheRatio, ONE - td.valueRatio);
48  }
```

Listing 3.1: `TrancheHelpers::bondCDR()`

Recommendation Improve the above routine by avoiding the possible divide-by-zero issue.

Status This issue has been fixed by this commit: `73008db`.

3.2 Incorrect incrementTrancheBal() Logic in PerpHelpers

- ID: PVE-002
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `PerpHelpers`
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [3]

Description

The `2Factor Finance` protocol has another utility `PerpHelpers` contract for perp reserve computations. This utility contract has a helper function to increment a tranche balance (and value) on a reserve snapshot. Our analysis shows this helper function does not properly update the reserve value (though its balance is correctly updated).

To elaborate, we show below the implementation of this related routine, i.e., `incrementTrancheBal()`. As the name indicates, this routine increments an input tranche balance and its value. It comes to our attention that the balance is properly updated (line 202). But its value should be updated as `rd.trancheData[i].value += rtd.data.value(rtd.balance)`, not current `rd.trancheData[i].value = rtd.data.value(rtd.balance)` (line 203).

```
194   function incrementTrancheBal(
195     IPerp.ReserveData memory rd,
196     TokenAmount memory trAmt
197   ) internal pure {
198     uint8 nTrs = uint8(rd.trancheData.length);
199     for (uint8 i = 0; i < nTrs; ++i) {
200       IPerp.ReserveTrancheData memory rtd = rd.trancheData[i];
201       if (address(rtd.data.tranche) == address(trAmt.token)) {
202         rd.trancheData[i].balance += trAmt.amount;
203         rd.trancheData[i].value = rtd.data.value(rtd.balance);
204         return;
205       }
206     }
207
208     revert UnexpectedAsset();
209   }
```

Listing 3.2: `PerpHelpers::incrementTrancheBal()`

Recommendation Revise the above `incrementTrancheBal()` routine to properly update a given tranche's balance and value.

Status This issue has been fixed by this commit: 0156381.

3.3 Enhanced redeemImmature() Logic in BondsManager

- ID: PVE-003
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: BondsManager
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [3]

Description

All bonds issued in 2Factor Finance are managed with a dedicated BondsManager contract. By managing the lifecycle of two-tranche (Senior/Junior)bonds as ERC-20 underlying assets, it greatly facilitates their creation, deposits, settlement at maturity, and redemptions. While reviewing the redemption logic over immature bonds, we notice an issue that does not properly result in the intended redeemed amounts.

In the following, we show the implementation of the related function, i.e., redeemImmature(). It has a rather straightforward logic in computing proportional senior and junior token amounts to be redeemed and underlying returned. In a corner case when the underlying return amount is zero (line 499), the redeemed amount of senior and junior token amounts should also be zero, not current srTrAmt and jrTrAmt (line 500).

```
464   function redeemImmature(
465     bytes32 bondID,
466     uint256 srAmtAvail,
467     uint256 jrAmtAvail
468   )
469     public
470     override
471     onlyValidBond(bondID)
472     onlyNotMature(bondID)
473     nonReentrant
474     returns (
475       uint256 underlyingAmtReturned,
476       TokenAmount memory srTrAmt,
477       TokenAmount memory jrTrAmt
478     )
479   {
480     BondData storage bd = bondsData[bondID];
481     if (srAmtAvail <= 0 || jrAmtAvail <= 0) {
482       srTrAmt = TokenAmount(bd.senior, 0);
483       jrTrAmt = TokenAmount(bd.junior, 0);
484       return (0, srTrAmt, jrTrAmt);
485     }
```

```
487     // Compute proportional sr and jr token amounts to be redeemed and underlying
            returned
488     uint256 srSupply = bd.senior.totalSupply();
489     uint256 jrSupply = bd.junior.totalSupply();
490     uint256 srAmtRedeemed = srAmtAvail;
491     uint256 jrAmtRedeemed = jrSupply.mulDiv(srAmtAvail, srSupply);
492     if (jrAmtRedeemed > jrAmtAvail) {
493       srAmtRedeemed = srSupply.mulDiv(jrAmtAvail, jrSupply);
494       jrAmtRedeemed = jrAmtAvail;
495     }

497     // Set return values
498     underlyingAmtReturned = bd.underlyingBal.mulDiv(srAmtRedeemed, srSupply);
499     srTrAmt = TokenAmount(bd.senior, srAmtRedeemed);
500     jrTrAmt = TokenAmount(bd.junior, jrAmtRedeemed);
501     if (underlyingAmtReturned <= 0) {
502       return (underlyingAmtReturned, srTrAmt, jrTrAmt);
503     }

505     // Perform bond book keeping
506     bd.underlyingBal -= underlyingAmtReturned;

508     // Burn sr and jr tranche tokens
509     bd.senior.burn(msg.sender, srTrAmt.amount);
510     bd.junior.burn(msg.sender, jrTrAmt.amount);

512     // Transfer underlying tokens back to the user
513     bd.underlying.safeTransfer(msg.sender, underlyingAmtReturned);
514     ...
515   }
```

Listing 3.3: `BondsManager::redeemImmature()`

Recommendation Improve the above routine to properly return zero redeemed senior and junior token amounts when the underlying token amount is zero.

Status This issue has been fixed by this commit: `ed1e758`.

3.4 Improved Reserve Tranche Data Cloning Logic in PerpHelpers

- ID: PVE-004
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: PerpHelpers
- Category: Coding Practices [5]
- CWE subcategory: CWE-1126 [1]

Description

As mentioned earlier, 2Factor Finance has a utility PerpHelpers contract for perp reserve computations. This utility contract has an internal helper function to clone a reserve tranche data array with a specified length. Our analysis shows this helper function may have an out-of-array-boundary issue that should be addressed.

To elaborate, we show below the implementation of this helper function, i.e., _cloneReserveTrancheData(). As the name indicates, it is used to clone a reserve tranche data array based on the given input len argument. However, the internal for-loop (line 240) does not validate the len and may lead to write outside the rtd_ array boundary (line 240). To fix, we need to improve the ending condition of the for as i < nTrs && i < len (line 240).

```
234  function _cloneReserveTrancheData(
235    IPerp.ReserveTrancheData[] memory rtd,
236    uint8 len
237  ) internal pure returns (IPerp.ReserveTrancheData[] memory rtd_) {
238    rtd_ = new IPerp.ReserveTrancheData[](len);
239    uint8 nTrs = uint8(rtd.length);
240    for (uint8 i = 0; i < nTrs; ++i) {
241      IPerp.ReserveTrancheData memory td = rtd[i];
242      rtd_[i] = IPerp.ReserveTrancheData({
243        data: td.data,
244        balance: td.balance,
245        value: td.value
246      });
247    }
248  }
```

Listing 3.4: PerpHelpers::_cloneReserveTrancheData()

Recommendation Improve the above routine by avoiding possible out-of-boundary array access.

Status This issue has been fixed by this commit: a3550dc.

3.5 Revisited Tranche Valuation Calculation Logic

- ID: PVE-005
- Severity: Low
- Likelihood: Low
- Impact: Low
- Target: `TrancheHelpers`
- Category: Business Logic [6]
- CWE subcategory: CWE-770 [3]

Description

As mentioned earlier, the utility `TrancheHelpers` contract in `2Factor Finance` is designed to facilitate tranche value calculation. Specifically, for a given amount of tranche tokens, it computes the tranche value based on a pre-fetched tranche data snapshot. Our analysis shows we need to revisit a specific corner case in current implementation.

In the following, we show the implementation of the related function, i.e., `value()`. The corner case occurs when `bondClaim == 0`, i.e., when the related tranche is not mature and the counterpart tranche has withdrawn all underlying tokens. In this specific corner case, the value of any given token amount should be simply zero, instead of current `amount.mulDiv(ONE, td.underlyingOpenPrice)` (line 30).

```
25  function value(
26    ITranche.TrancheData memory td,
27    uint256 amount
28  ) internal pure returns (uint256) {
29    if (td.bondClaim <= 0) {
30      return amount.mulDiv(ONE, td.underlyingOpenPrice);
31    }
32    return td.trancheClaim.mulDiv(amount, td.trancheSupply);
33  }
```

Listing 3.5: `TrancheHelpers::value()`

Recommendation Improve the above-mentioned routine to properly calculate the value of the given tranche token amounts.

Status This issue has been fixed by this commit: `67f21d1`.

3.6 Trust Issue of Admin Keys

- ID: PVE-006
- Severity: Medium
- Likelihood: Low
- Impact: Medium
- Target: `Multiple Contracts`
- Category: Security Features [4]
- CWE subcategory: CWE-287 [2]

Description

In the `2Factor Finance` protocol, there is a special administrative account, i.e., `owner`. This `owner` account plays a critical role in governing and regulating the system-wide operations (e.g., configure various settings and execute privileged operations). It also has the privilege to control or govern the flow of assets within the protocol contracts. In the following, we examine the privileged account and the related privileged accesses in current contracts.

```
208   function updateIssuer(IBondIssuer issuer_) external onlyOwner {
209     if (address(issuer) != address(0)) {
210       perpJr.revokeApproval(address(issuer.bondsManager()));
211     }
212     perpJr.primeApproval(address(issuer_.bondsManager()));
213     issuer = issuer_;
214   }

216   ...
217   function updateFeePolicy(IFeePolicy feePolicy_) external onlyOwner {
218     feePolicy = feePolicy_;
219   }

221   ...
222   function updatePerpBalancer(IPerpBalancer balancer_) external onlyOwner {
223     balancer = balancer_;
224   }

226   ...
227   function updateRolloverEligibility(
228     uint32 minMaturitySec_
229   ) external onlyOwner {
230     if (minMaturitySec_ <= 0) {
231       revert InvalidParamDefn();
232     }
233     minMaturitySec = minMaturitySec_;
234   }

236   ...
237   function updateDRTargets(
238     uint256 targetSystemRatio_,
239     uint256 targetPerpJrVM_
```

```
240   ) external onlyOwner {
241     uint64 jrTrancheRatio = (uint64(ONE) - issuer.srTrancheRatio());
242     uint256 minSystemRatio = ONE.mulDiv(jrTrancheRatio, ONE - jrTrancheRatio);
243     if (targetSystemRatio_ <= minSystemRatio || targetSystemRatio_ <= 0) {
244       revert InvalidParamDefn();
245     }

247     uint256 maxJrVM = ONE.mulDiv(ONE, jrTrancheRatio);
248     if (targetPerpJrVM_ >= maxJrVM || targetPerpJrVM_ <= 0) {
249       revert InvalidParamDefn();
250     }

252     targetSystemRatio = targetSystemRatio_;
253     targetPerpJrVM = targetPerpJrVM_;
254   }

256   ...
257   function updateMinValRolled(uint256 minValRolled_) external onlyOwner {
258     minValRolled = minValRolled_;
259   }
```

Listing 3.6: Example Privileged Operations in `PerpController`

We understand the need of the privileged functions for proper contract operations, but at the same time the extra power to these privileged accounts may also be a counter-party risk to the contract users. Therefore, we list this concern as an issue here from the audit perspective and highly recommend making these privileges explicit or raising necessary awareness among protocol users.

In the meantime, the audited protocol makes use of the proxy contracts to allow for future upgrades. The upgrade is a privileged operation and the management of the related admin key also falls in this trust issue.

Recommendation Promptly transfer the privileged account to the intended DAO-like governance contract. All changes to privileged operations may need to be mediated with necessary timelocks. Eventually, activate the normal on-chain community-based governance life-cycle and ensure the intended trustless nature and high-quality distributed governance.

Status The issue has been resolved as the team clarifies the use of a publicly documented multisig address as the `owner` account.

4 Conclusion

In this audit, we have analyzed the design and implementation of the 2Factor Finance protocol for perpetual tranches. The protocol creates senior and junior tranches from underlying assets, wrapped as perpetual ERC20 notes with automatic rollover and rebalancing mechanics. It enables users to gain leveraged or de-leveraged exposure to underlying assets through a two-tranche bond system. Users can mint perpetual senior notes for lower-volatility exposure or perpetual junior notes for leveraged exposure, backed by dynamically managed reserves of tranched bonds. The current code base is clearly organized and those identified issues are promptly confirmed and fixed.

Meanwhile, we need to emphasize that Solidity-based smart contracts as a whole are still in an early, but exciting stage of development. To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

PeckShield

References

[1] MITRE. CWE-1126: Declaration of Variable with Unnecessarily Wide Scope. https://cwe.mitre.org/data/definitions/1126.html.

[2] MITRE. CWE-287: Improper Authentication. https://cwe.mitre.org/data/definitions/287.html.

[3] MITRE. CWE-770: Allocation of Resources Without Limits or Throttling. https://cwe.mitre.org/data/definitions/770.html.

[4] MITRE. CWE CATEGORY: 7PK - Security Features. https://cwe.mitre.org/data/definitions/254.html.

[5] MITRE. CWE CATEGORY: Bad Coding Practices. https://cwe.mitre.org/data/definitions/1006.html.

[6] MITRE. CWE CATEGORY: Business Logic Errors. https://cwe.mitre.org/data/definitions/840.html.

[7] MITRE. CWE VIEW: Development Concepts. https://cwe.mitre.org/data/definitions/699.html.

[8] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology.

[9] PeckShield. PeckShield Inc. https://www.peckshield.com.